

Hackathon Web Transactionnelle

Plateforme de Gestion de Ligues Sportives Communautaires

Checkpoint de progression – Séance 08h00 à 11h00

Durée maximale du hackathon : 48 heures

Objectif de cette séance

Vous êtes déjà en plein hackathon. Cette séance n'est donc pas un cours magistral classique. Elle sert à vérifier votre avancement, clarifier votre MVP, débloquer les problèmes techniques et vous pousser vers une version fonctionnelle du projet.

Votre objectif n'est pas de tout faire. Votre objectif est de livrer une version utilisable, cohérente, testable et présentable de la plateforme.

Rappel du sujet

Vous devez développer une plateforme web moderne permettant de gérer des ligues sportives communautaires. L'application connecte principalement deux types d'utilisateurs :

- des **organiseurs**, qui créent et gèrent des tournois, des équipes, des demandes et des matchs ;
- des **joueurs**, qui recherchent des équipes, envoient des demandes d'adhésion et suivent leurs matchs.

L'idée générale est de créer une sorte de plateforme de rencontre sportive communautaire, comparable à un « Meetup » pour les sports amateurs.

1. Informations générales de l'équipe

Nom du projet	Meetup Sportif
Nom de l'équipe	FullStack FC
Membres de l'équipe	Sonia Corbin, Marina Kamel
Répartition des rôles	Frontend : Sonia Corbin Backend : Marina Kamel BDD / Prisma : Sonia Corbin, Marina Kamel Auth / Stripe : Sonia Corbin, Marina Kamel
Lien GitHub	https://github.com/MarinaKamel-coder/meetup.git

2. Stand-up rapide de début de séance

Chaque équipe doit répondre rapidement aux questions suivantes.

Question	Réponse de l'équipe
Qu'avez-vous fait hier entre 14h et 17h ?	Configuration complète du projet (Clerk, Neon, Prisma, middleware), toutes les Server Actions backend, schéma Prisma complet, et toutes les pages frontend avec redesign visuel glassmorphism.
Quelle partie du projet est déjà commencée ?	Tout le cœur du MVP : Authentification, Dashboards (Joueur, Orga, Admin), et la logique de demande d'adhésion.
Qu'est-ce qui bloque actuellement l'équipe ?	Intégration Stripe à tester en conditions réelles (flux tournoi payant) et on a réglé un blocage de CSP (Content Security Policy) qui empêchait Clerk et Stripe de charger. mais c'est débloqué maintenant.
Quelle fonctionnalité voulez-vous rendre visible avant 11h ?	Scénario complet : organisateur crée un tournoi → crée une équipe → joueur envoie une demande → organisateur accepte → joueur rejoint l'équipe.

Règle du hackathon

Une fonctionnalité non visible, non testable ou non démontrable ne compte pas vraiment. À partir de maintenant, vous devez penser en termes de démonstration : qu'est-ce que vous pouvez réellement montrer ?

3. MVP obligatoire : checklist de progression

Cochez ce qui est déjà fait, puis indiquez ce qui reste à faire.

Fonctionnalité MVP	Statut	Notes / Problèmes / Prochaines étapes
Authentification avec Clerk	☒	Fonctionnel — sign-in, sign-up, redirection par rôle
Synchronisation User Clerk vers Prisma	☒	Webhook /api/webhooks/clerk avec vérification Svix
Gestion des rôles : PLAYER, ORGANIZER, ADMIN	☒	Rôles PLAYER, ORGANIZER, ADMIN fonctionnels.

Profil joueur : ville, sport, niveau, poste	☒	Page /profile avec formulaire React Hook Form
Dashboard organisateur	☒	Stats dynamiques Prisma (tournois, équipes, demandes)
Création et gestion des tournois	☒	CRUD complet avec Server Actions + validation Zod
Création et gestion des équipes	☒	createTeam() avec vérification organisateur
Recherche d'équipes côté joueur	☒	Page /teams avec cards et progress bar de remplissage
Demandes d'adhésion	☒	createJoinRequest(), cancelJoinRequest()
Acceptation / refus des demandes	☒	acceptJoinRequest(), rejectJoinRequest() dans prisma.\$transaction
Matches : création, liste, modification simple	☒	createMatch(), updateMatchScore()
Païement Stripe pour tournoi payant	☐	En cours / Partiel, Le champ entryFee est intégré au modèle Tournament. La logique de redirection vers Checkout est la prochaine priorité.
Section admin custom	☒	Page /admin réservée au rôle ADMIN

4. Priorité absolue : la fonctionnalité centrale

Fonctionnalité centrale du projet

La fonctionnalité la plus importante du sujet est le système de **demandes d'adhésion**. C'est elle qui donne une vraie logique métier à l'application.

Le scénario principal est le suivant :

1. Un organisateur crée un tournoi.
2. L'organisateur crée une ou plusieurs équipes dans ce tournoi.
3. Un joueur recherche une équipe disponible.
4. Le joueur envoie une demande d'adhésion.
5. Si le tournoi est payant, le joueur doit passer par Stripe.
6. L'organisateur voit les demandes reçues.
7. L'organisateur accepte ou refuse la demande.
8. Si la demande est acceptée, le joueur devient membre de l'équipe.

Votre état actuel sur cette fonctionnalité

Question	Réponse
Avez-vous déjà créé le modèle JoinRequest dans Prisma ?	Oui — avec status, paymentStatus, stripeSessionId, paidAt
Avez-vous empêché les demandes en double ?	oui — contrainte @@unique([playerId, teamId]) dans Prisma
Avez-vous vérifié la capacité maximale d'une équipe ?	Oui — vérification members.length < maxCapacity dans la transaction
Avez-vous une Server Action createJoinRequest() ?	Oui — avec auth check et gestion paymentStatus
Avez-vous une Server Action acceptJoinRequest() ?	Oui — avec vérification rôle ORGANIZER
L'acceptation se fait-elle dans une transaction Prisma ?	Oui — prisma.\$transaction avec vérification capacité + connect membre
Le joueur voit-il le statut de ses demandes ?	Oui — page /my-requests avec badges PENDING/ACCEPTED/REJECTED

L'organisateur voit-il les demandes reçues ?	Oui — page /requests avec boutons Accepter/Refuser
--	--

5. Modèle de données : vérification rapide

Votre base de données doit au minimum contenir les éléments suivants.

- User ✓
- PlayerProfile ✓
- Tournament ✓
- Team ✓
- JoinRequest ✓
- Match ✓

État du schéma Prisma

Modèle	Créé ?	Notes
User	Oui / Non / Partiel	clerkId, email, fullName, role (PLAYER/ORGANIZER/ADMIN)
PlayerProfile	Oui / Non / Partiel	city, favoriteSport, level, position — relation 1 :1 avec User
Tournament	Oui / Non / Partiel	name, sport, city, startDate, entryFee, currency, organizerId
Team	Oui / Non / Partiel	name, tournamentId, maxCapacity — relation N :M avec User
JoinRequest	Oui / Non / Partiel	playerId, teamId, status, paymentStatus, stripeSessionId, paidAt
Match	Oui / Non / Partiel	teamAId, teamBId, date, location, scoreA, scoreB

6. Pages et routes à viser

Page / Route	Rôle	Statut
/	Landing publique	✓

/	Landing publique	✓
/sign-in et /sign-up	Authentification Clerk	✓
/profile	Profil joueur	✓
/dashboard	Dashboard organisateur	✓
/tournaments	Liste / gestion des tournois	✓
/tournaments/[id]	Détail d'un tournoi	✓
/teams	Recherche d'équipes	✓
/teams/[id]	Détail d'une équipe	✓
/my-requests	Demandes du joueur	✓
/requests	Demandes reçues par l'organisateur	✓
/matches	Matches	✓
/admin	Administration custom	✓

7. Server Actions et Route Handlers

Server Actions prioritaires

- `updatePlayerProfile()` ✓
- `createTournament()` ✓
- `updateTournament(id)` ✓
- `deleteTournament(id)` ✓
- `createTeam()` ✓
- `createJoinRequest()` ✓

- cancelJoinRequest(id) ✓
- acceptJoinRequest(id) ✓
- rejectJoinRequest(id) ✓
- createMatch() ✓
- updateMatchScore(id) ✓

Route Handlers nécessaires

- POST /api/webhooks/clerk ✓
- POST /api/webhooks/stripe ✓
- POST /api/checkout ✓

8. Stripe : décision immédiate

Attention

Si votre application permet de créer un tournoi avec des frais d'inscription supérieurs à 0, le paiement Stripe devient obligatoire dans le MVP.

Si vous êtes en retard techniquement, commencez d'abord par faire fonctionner tout le parcours avec un tournoi gratuit. Ensuite seulement, ajoutez le flux Stripe pour les tournois payants.

Question	Réponse
Avez-vous prévu des tournois payants ?	Oui / Non / Pas encore décidé
Le champ entryFee existe-t-il dans Tournament ?	Oui — champ entryFee dans Tournament (en cents, 0 = gratuit)
Le champ paymentStatus existe-t-il dans JoinRequest ?	Oui — enum : NOT_REQUIRED, PENDING, PAID
La route /api/checkout est-elle commencée ?	Oui — création session Stripe Checkout avec metadata.joinRequestId
Le webhook Stripe est-il commencé ?	Oui — vérification signature + mise à jour paymentStatus à PAID
Avez-vous testé Stripe CLI ?	Partiellement — flux gratuit testé, flux payant en cours de validation

9. Objectif concret avant 11h00

Chaque équipe doit choisir une seule priorité principale pour la fin de la séance.

À 11h00, votre équipe doit pouvoir montrer au moins une chose concrète :

- une page fonctionnelle ;
- une connexion réussie ;
- une création de tournoi ;
- une création d'équipe ;
- une demande d'adhésion ;
- une acceptation de demande ;
- une liste affichée depuis Prisma ;
- ou une autre fonctionnalité visible et testable.

Élément	Réponse de l'équipe
Fonctionnalité à démontrer avant 11h00	Scénario complet de demande d'adhésion : organisateur crée tournoi → équipe → joueur envoie demande → organisateur accepte → joueur devient membre
Responsable principal de cette fonctionnalité	Sonia Corbin (frontend) + Marina Kamel (backend)
Ce qui est déjà fait	: Schéma Prisma complet (PostgreSQL/Neon), Authentification Clerk avec rôles RBAC, Server Actions transactionnelles pour les demandes d'adhésion (create, accept, reject), et UI réactive
Ce qui reste à faire	Tester le flux Stripe en conditions réelles
Risque principal	Webhook Stripe non validé en local
Plan B si cela ne fonctionne pas	Démontrer avec un tournoi gratuit (flux sans Stripe)

10. Priorisation : Must Have, Nice to Have, Abandon

Must Have

Ce qui doit absolument être fait pour que le projet soit présentable.

- Scénario complet demande d'adhésion fonctionnel
- Authentification Clerk avec redirection par rôle
- Dashboard organisateur avec stats réelles

- Pages joueur (teams, profile, my-requests) fonctionnelles

Nice to Have

Ce qui est intéressant seulement si le MVP fonctionne déjà.

- Flux Stripe complet pour tournois payants
- Déploiement Vercel
- Export CSV depuis l'admin

À abandonner si le temps manque

Ce qui peut être retiré sans détruire la logique principale du projet.

- Notifications temps réel
- Système de classement automatique
- Messagerie entre joueurs

11. Pièges techniques à éviter

- Ne pas importer Prisma dans un Client Component.
- Ne pas oublier `revalidatePath` après une mutation.
- Ne jamais faire confiance aux données envoyées par le client.
- Toujours valider les entrées côté serveur avec Zod.
- Ne jamais exposer les secrets dans `NEXT_PUBLIC_*`.
- Ne pas vérifier l'auth uniquement dans le middleware : vérifier aussi dans les Server Actions sensibles.
- Ne pas passer trop de temps sur le design avant d'avoir une logique fonctionnelle.
- Ne pas commencer les bonus avant d'avoir terminé le MVP.

12. Mini pitch technique

À la fin de la séance, chaque équipe doit être capable de présenter rapidement son avancement.

Notre projet s'appelle : Meetup Sportif

Notre plateforme permet de : Connecter organisateurs et joueurs autour de tournois sportifs communautaires

Notre utilisateur joueur peut : Rechercher des équipes, envoyer des demandes d'adhésion, suivre ses demandes et voir ses matchs

Notre utilisateur organisateur peut : Créer des tournois et équipes, gérer les demandes d'adhésion, planifier des matchs et consulter son dashboard

Notre fonctionnalité centrale est : Le système de demandes d'adhésion avec workflow d'approbation et transaction Prisma sécurisée

Notre base de données contient actuellement : 6 modèles : User, PlayerProfile, Tournament, Team, JoinRequest, Match

La fonctionnalité que nous pouvons démontrer maintenant est : Scénario complet : création tournoi → équipe → demande → acceptation

Ce qui nous reste à faire en priorité est : Validation du flux Stripe pour les tournois payants

13. Feedback de l'enseignant

Critère	Feedback
Clarté de l'idée	A REMPLIR
Avancement technique	A REMPLIR
Qualité du modèle de données	A REMPLIR
Respect du MVP	A REMPLIR
Organisation de l'équipe	A REMPLIR
Prochaine action recommandée	A REMPLIR

14. Message final

Un hackathon ne se gagne pas avec la quantité d'idées. Il se gagne avec la capacité à choisir, construire, tester et présenter.

Un petit parcours complet qui fonctionne vaut mieux qu'une grande application incomplète.

Priorisez le scénario principal :

Organisateur crée un tournoi → crée une équipe → joueur demande à rejoindre → organisateur accepte → joueur rejoint l'équipe

Si ce scénario fonctionne, votre projet possède déjà un coeur solide.

Clarifiez. Priorisez. Codez. Testez.

Bon courage pour la suite du hackathon.